

The Collection: technical guide

For engineers who build with the collection, contribute to it, or run it. It starts from the basics and ends at deployment.

For	AI champions and other engineers; platform and integration engineers; operators
Date	20 September 2026
Status	Reflects the repository at the date above; the readiness page in the repository is the live record

1. The basics, if you're new to this

A language model is a function from text to text. You give it a prompt, it gives you a continuation. It has no memory between calls, no access to anything you don't put in the prompt, and no way to act on the world. Everything interesting, and everything risky, comes from what we wrap around it.

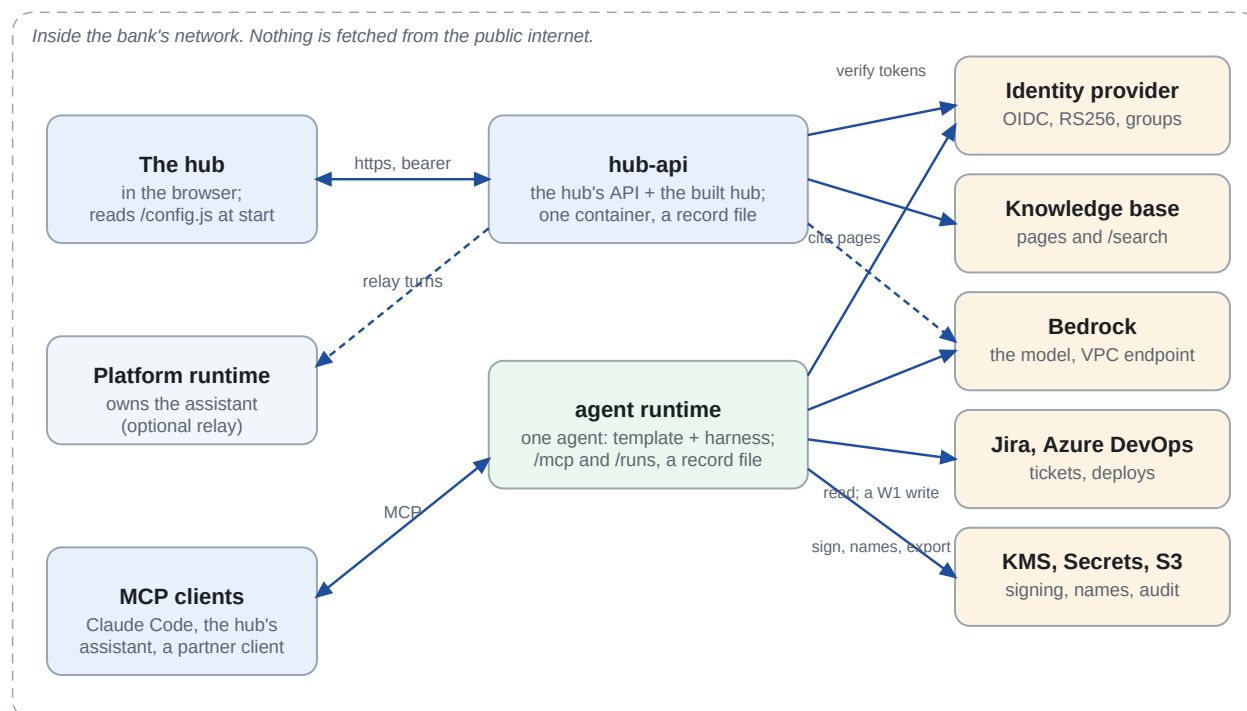
An **assistant** is a model wrapped in a loop that fetches relevant pages, puts them in the prompt, and shows the answer. It reads, it doesn't act. An **agent** is a model wrapped in a loop that can also call tools: read a ticket, look up a deploy, post a comment. The moment a model can call tools, the question that matters is not how clever the model is but what the loop lets it do, and who has to say yes first.

That loop is what we call the **harness**. It's ordinary code, it runs the same way every time, and the model cannot change it. When people in this repository say an agent is safe to run, they mean the harness is, and they can point at the test that proves each rule. Keep that distinction in mind and the rest of this guide will read easily.

One more term. **Prompt injection** is when text the model reads (a ticket, a web page, a tool description) contains instructions, and the model follows them as if they came from you. The defence here is simple to state: text the model reads is evidence, never instruction. Sources are fenced off, scored, and if one looks like an instruction the session is marked tainted and can only read from then on.

2. What's in the repository

One repository, four parts. **components/** is the shelf: 29 building blocks, each in its own directory with a manifest, a README that gets you running in five minutes, a step-by-step walkthrough, a live example and tests. **hub/** is the employee AI hub front end. **services/** holds the two things we deploy: hub-api, the hub's back end, and agent-runtime, which runs one agent. **tools/** holds the shelf tool that checks every manifest and generates the hub's listings and the knowledge base's pages. The knowledge base itself is a separate product; we publish pages into it.



Where each piece runs inside the bank and what it talks to. The two services are one container each with a record file on a volume.

3. The shelf: what a component is

Every component declares a **category**, and the category tells you who it's for and what it has to contain. A **skill** is a written procedure with a template and a filled example; anyone can follow it and there's nothing to run. A **tool** is code with one clear surface and a test. An **integration** is a client for an external system that ships with an in-memory fake behind the same methods, so you can test without an account. A **pattern** is a small reference implementation of a practice. A **harness** is the loop an agent runs inside. An **agent** is the composite: a template that says what it is and what it may call, the tools it calls, and the harness it runs in.

The manifest, `component.json`, carries the version, the two sign-off slots, where it was lifted from, its tags, the test command, the walkthrough and example paths, any files vendored from another component, and which sections of the platform design specification it implements in the meantime, with the test the platform has to pass before the component can be retired. The shelf tool checks all of it on every commit.

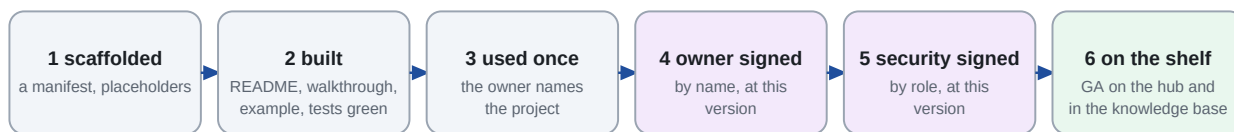
The rules you'll bump into

Python components use the standard library only, unless the manifest says otherwise. A component imports nothing outside its own directory; when two need the same file, one owns it and the other vendors a byte-identical copy and declares it. Credentials are names looked up at call time, never values. And any change a consumer would notice bumps the version, which makes both sign-offs stale until they're given again.

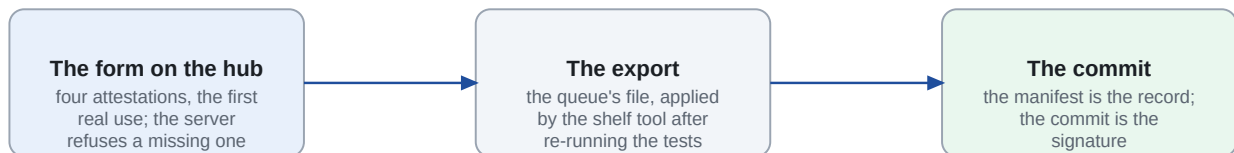
Adding one

```
python3 tools/new_component.py python my-tool --category tool --summary "What it does"
python3 tools/new_component.py agents my-agent --category agent --summary "..."
python3 tools/shelf.py --write && python3 tools/shelf.py --test --only python
```

For an agent, write `TEMPLATE.md` first and then one test per line of its never list; the incident first-read agent is the model to copy. When the tests are green and it has run once for real, the owner and an AI security engineer sign it on the hub, an engineer applies the export with `python3 tools/shelf.py --apply-signoffs` and commits, and the commit is the signature.



A version bump returns a component to stage 3: both people sign the new version.

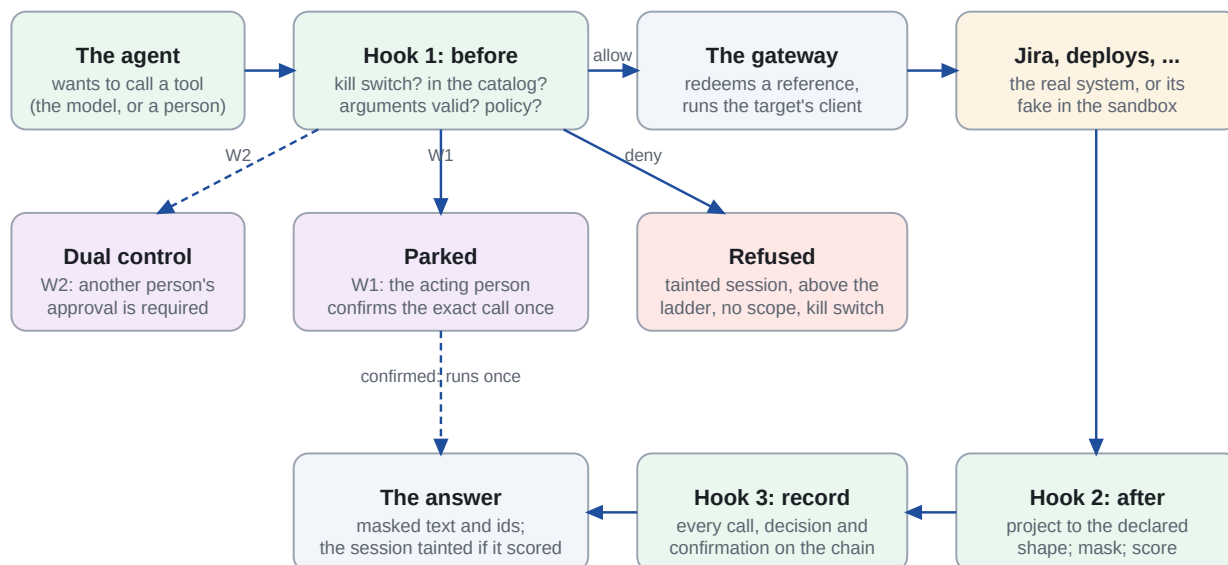


The six stages, each read from the manifest rather than guessed, and the path from the form on the hub to the commit.

4. How an agent runs

The **governed-action-loop** component is the harness. Every call an agent makes goes through three hooks in a fixed order, and there is no way to reach a target around them.

One call through the loop. Reads run; a write waits for the acting person; a bigger change needs a second person; money is forbidden.



One call through the loop.

Before the call, the harness checks for a kill switch, checks the tool is in the signed catalog, validates the arguments, and asks the policy bundle. A read runs. A write (W1) is parked with a hash of the exact tool and arguments until the acting person confirms it; the confirmation is consumed once, so a replay fails. A bigger change (W2) needs another person's approval reference. Money is forbidden outright. After the call, the result is projected to the shape the catalog declared, masked for personal data, and scored; if it scores as an injection the session is tainted and capped at reads. And every call, decision, confirmation and stop is written to a hash-chained record you can verify.

The **incident-first-read-agent** is the reference. Its `TEMPLATE.md` is data: the role, the stages, the tools with their tiers and result shapes, a budget, and a list of things it never does. The code builds the harness's signed catalog from that list, so the agent can call exactly what the template names and nothing else. Its tests include one per line of the never list. Run `python3 example.py` in its directory to watch a clean incident and then a poisoned one.

5. MCP, in one paragraph each

MCP is the protocol AI clients use to call tools. Three components speak it. **mcp-tool-server** puts the harness behind an MCP transport: the tool list is the signed catalog, every call runs the hooks, a W1 call becomes an elicitation the client puts to the person, and a tainted or out-of-scope call is a 403 that names the scope. **mcp-gateway-client** is the other direction: the harness calling someone else's MCP server under a contract record, with the tool descriptions pinned and any drift quarantining the server. **shelf-mcp-server** is read-only and lets a coding assistant such as Claude Code browse the shelf without cloning it.

6. The services

hub-api implements the hub's whole API on the Python standard library. It verifies the bank's RS256 tokens against the identity provider's key set, maps directory groups to roles and entitlements with a file the platform team owns, serves the catalog from the bank's own listings plus the shelf's components,

keeps briefs, requests and sign-offs in a SQLite record, and streams assistant turns from one of three back ends: a fake for development, a relay to the platform runtime, or Bedrock with the knowledge base's search for sources. It also serves the built hub itself and a small config.js the page reads at start, so the hub is built once and configured per environment.

The hub's guide is part of hub-api too. tools/shelf.py --write splits the repository's pages (the root pages, the knowledge-base pages and every component README) into a corpus of a few hundred passages, each tagged with the audience it was written for. POST /guide/ask screens the question for instructions with the harness's guard, ranks the passages with BM25 and a small table of plain-language synonyms, and answers either with the passages themselves, attributed, or, when HUB_ASSISTANT is bedrock, with a cite-or-drop answer from the model over those passages only. The hub's in-browser mock ranks the same corpus the same way, so the guide works in the sandbox without a service.

agent-runtime serves one agent of the collection over MCP and a small run API. Everything that touches a bank system is chosen by configuration: the identity provider or a fake, a KMS key or a local one, Bedrock or the offline rules engine, the Jira and Azure DevOps connectors or their fakes. Staging and production refuse every fake and require a real system for every target the agent's template names. The record is verified at every start and exported to S3.

7. Configuring it for the bank

Every variable is in config/collection.env.example, and CONFIGURATION.md explains what the bank has to supply for each system. Two conventions carry most of the weight. A variable ending in _NAME is the name of a secret in the secrets provider, never a value; the task role's SigV4 credentials fetch it at call time. And every service's settings fail closed: check-config lists each problem by variable name and exits, and the container's entrypoint runs it before the process listens. The same image runs in the sandbox with fakes and in production against the bank; only the environment changes.

System	What the bank supplies
Identity provider	An app registration per deployable with the groups claim on tokens; the group ids for the identity map; the redirect URI
Model	An inference profile approved by model risk; the VPC endpoint; InvokeModel on the task role
Secrets and keys	Secrets under the service prefix; an asymmetric KMS key for catalog signing
Jira, Azure DevOps	A service account with read on incident projects and comment; a PAT scoped to pipelines
Knowledge base	The console URL and its /search endpoint
Storage	An EFS volume for each record; a KMS-encrypted bucket for the audit export

8. Deploying it

Build the two images from the repository root with the base image digest pinned to the bank's approved one; the hub's dist is prebuilt and needs no rebuild for a new environment. Mount identity-map.json and consumers.json on the hub's config volume, create the secrets, apply the task roles from services/*/deploy/iam-task-role-policy.json, register the task definitions, and check /api/health and /health. Locally, docker compose -f deploy/compose.yaml up runs both with fakes; without a container daemon, scripts/smoke-container-tree.sh assembles each Dockerfile's copy set and starts it.

```
scripts/verify.sh python # the shelf, the vendoring, every Python component and service,
the refusals, the container trees
```

```
scripts/verify.sh hub      # typecheck, lint, tests, build
scripts/bundle.sh          # the offline release with the hub prebuilt and a sha256 manifest
```

Both CI definitions, GitHub Actions and Azure Pipelines, call the same verify script. The bundle unpacked in a clean directory passes every Python gate with no network and no Node.

9. Running it

RUNBOOK.md is the page to have open. It covers what a refused start looks like, rolling out a build, applying a sign-off, changing listings or grants, stopping the agent, rotating a credential, and exporting the chain. One rule from it is worth repeating here: if the record doesn't verify, don't repair rows. Restore the volume, compare the chain's head with the last export, and open an incident. SECURITY.md maps each threat to the control and the test that covers it.

10. Where the platform takes over

None of this is meant to outlive the platform. Each component's manifest names the sections of the platform design specification it implements in the meantime and a replacement test: the condition the platform has to meet before the component is retired. The MCP tool server is the specification's R1 road with the harness behind the transport; the gateway client is its contract-registry rules; the harness's stores and gateway are placeholders for the control layer. When those arrive, the tests are the handover.